JOHNS HOPKINS
WHITING SCHOOL of ENGINEERING

Introduction to Generative AI

Autoencoders and Applications

Outline

- Module Learning Objectives
- Introduction to Machine Learning
- Supervised Learning
- Unsupervised Learning
- Reinforcement Learning
- Neural Networks
- Variational Autoencoders

Module Learning Objectives

- Understand the paradigms of supervised, unsupervised, and reinforcement learning
- Apply supervised learning techniques to real-world problems, focusing on components such as datasets, models, algorithms, and loss functions
- Explore and analyze data using unsupervised learning methods like clustering, dimensionality reduction, and anomaly detection
- Develop skills in evaluating ML models using appropriate metrics like accuracy, precision, recall, and F1-score
- Understand the architecture, applications of deep neural networks, backpropagation
- Comprehend variational autoencoders (VAEs), probabilistic encoding/decoding

Introduction to Machine Learning (ML)

Supervised Learning

- The algorithm is trained on **labeled** dataset
- Intends to infer a **conditional** density $\mathbf{P}_X(x|y)$ on the labels y
- Learn a mapping from inputs to outputs to accurately predict or classify unseen data
- Image recognition, spam detection, sentiment analysis, fraud detection, credit scoring

Unsupervised Learning

- The algorithm is trained on **unlabeled** dataset
- Intends to infer an apriori density $\mathbf{P}_X(x)$
- Learn to identify data patterns or structures without explicit guidance
- Topic modeling, anomaly detection, document clustering, recommender systems, social network analysis

Reinforcement Learning

- A form of learning from experience through **reward** and **penalty**
- The **agent** intends to affect the **environment**, which can be stochastic and intractable large
- Learn to make decisions to maximize a reward by training **evaluation**
- Game playing, robotic control, autonomous systems, resource allocation, drug discovery

Learning Methodology

- **Training** the model to learn patterns by fitting to labeled data (training set) using regression, density fit, or optimization
- **Testing** the model performance is evaluated on unseen data (testing set) to measure its generalization ability
  - Can be model evaluation or before deployment (use the entire dataset, why?)
- **Cross-validation** the dataset is split into multiple non-overlapping subsets to repeatedly train and test the model, ensuring robust performance evaluation
  - Considered one of the most common and reliable evaluation methods
- **Model Evaluation** using metrics like accuracy, precision, recall, and F1-score to assess the model performance on the test data

Learning Methodology

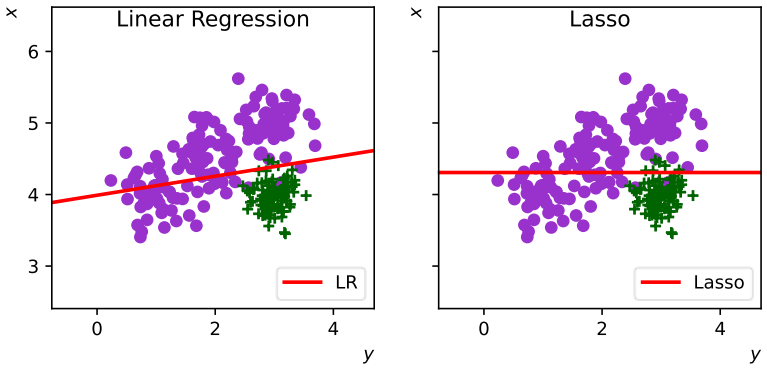
- **Data Preprocessing** cleaning and transforming raw data (e.g., normalization, handling missing values) to prepare it for model training
- **Feature Engineering** creating or selecting the most relevant features from the raw data to improve model accuracy and performance
- **Hyperparameter** tuning adjusting model parameters (e.g., learning rate, regularization) to optimize performance during the training phase
- **Model Selection** comparing different algorithms (e.g., decision trees, neural networks, SVMs) to find the best fit for the problem
- **Overfitting Prevention** techniques like regularization and dropout to prevent the model from memorizing the training data so it will generalize better

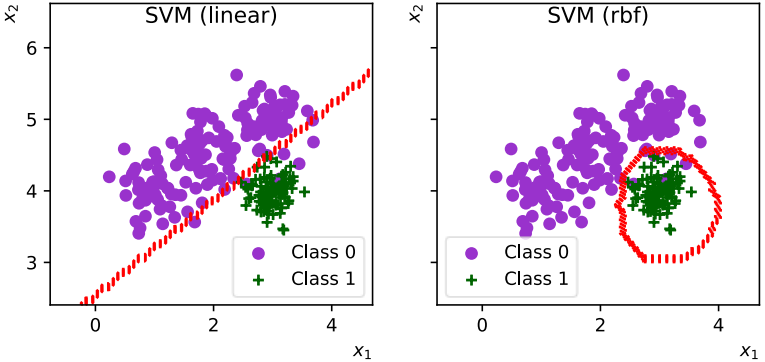
Supervised Learning

- **Classification** is predicting whether a test sample belongs to one of the categorical classes
- If there are only two classes, it is a **binary classification** problem
- If an algorithm can only work with binary classes, a set of **N-choose-2** classifiers is built, i.e. one for every class pair
- Minimizes an error (or maximizes a metric) where the error is the difference between the target output and the actual output

- **Regression** is the process when numerical values are predicted for a dependent variable
- Regression and supervised learning are complementary
- Model output is continuous, non-nominal, or real-valued
- The regression process can be considered a function approximation or a curve fit

Supervised Learning





- Linear Regression
  - Least squares to compute a hyperplane that minimizes residues
  - Prediction is by the fit hyperplane
- J= $\sum_i (\hat{y}_i - y_i)^2$

- Support Vector Machine
  - Large margin classifier, optimizes the distance between the decision surface hyperplane and the closest data points
  - Convex optimization, global max
- J= $\frac{1}{2}\|\mathbf{w}\|^2 + C\sum_i \max(0, 1 - y_i\,\mathbf{w}^{\mathrm{T}} x_i)$

Performance Metrics for Classification

- **Accuracy** The ratio of correctly predicted instances to the total instances
  - Can be misleading if the classes are unbalanced
- **Confusion Matrix** A table that summarizes the performance of a classification model
  - Shows the counts of true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN)

	Predicted Pos	**Predicted Neg**
Actual Pos	TP	FN
Actual Neg	FP	TN

- Accuracy= $\frac{\mathrm{TP+TN}}{\mathrm{TP+TN+FP+FN}}$
- P_D=TPR= $\frac{\mathrm{TP}}{\sum \mathrm{truth\ P}} = \frac{\mathrm{TP}}{\mathrm{TP+FN}}$
- P_F=FPR= $\frac{\mathrm{FP}}{\sum \mathrm{truth\ N}} = \frac{\mathrm{FP}}{\mathrm{FP+TN}}$

Performance Metrics

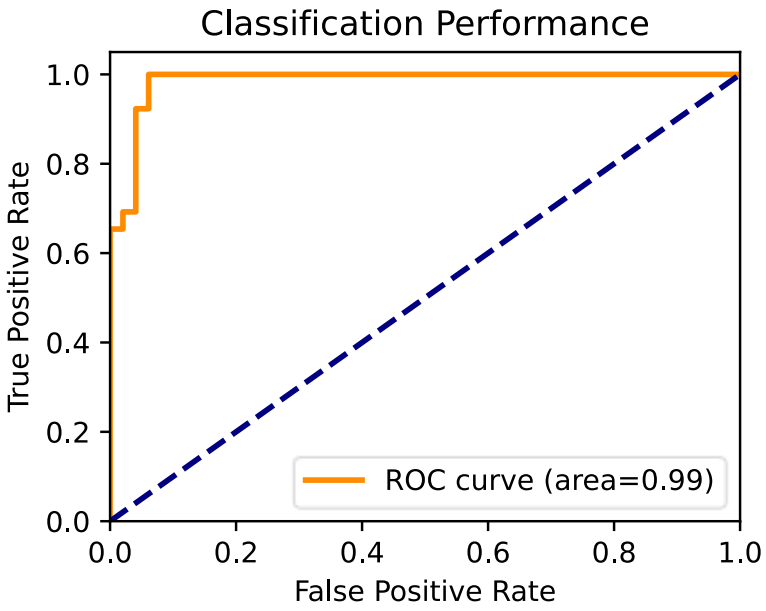
- **Precision** The ratio of correctly predicted positives among predicted positives
- Precision= $\frac{\mathrm{TP}}{\mathrm{TP+FP}}$

- **Sensitivity** The ratio of correctly predicted positives among actual positives
- Recall= $\frac{\mathrm{TP}}{\mathrm{TP+FN}}$

- **F1-Score** The harmonic mean of precision and recall
- Balances the two metrics and is useful when the class distribution is unbalanced
- F1-Score= $2\frac{\mathrm{Precision \cdot Recall}}{\mathrm{Precision+Recall}}$

Performance Metrics

- **The Receiver Operating Characteristic** The ROC curve plots the true positive rate (probability of detection) against the false positive rate (false alarm)
  - FPR= $1 - \frac{\mathrm{TN}}{\mathrm{TN+FP}}$
- **The Area Under the Curve** The AUC measures the probability that the classifier ranks a randomly chosen positive instance higher than a randomly chosen negative instance
  - An AUC of 1.0 indicates perfect
  - An AUC of 0.5 suggests random



Performance Metrics for Regression

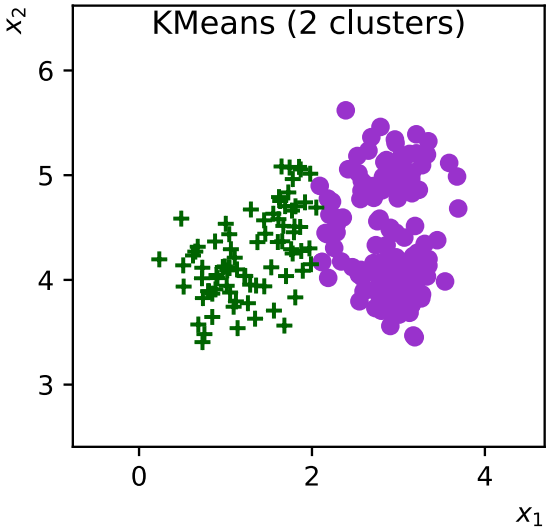
- **Mean Absolute Error** (MAE) The average of the absolute differences between the predicted values and the actual values
- MAE= $\frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i|$
- **Mean Squared Error** (MSE) The average of the squared differences between the predicted values and the actual values
- MSE= $\frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$
- **R-squared** (R^2) The proportion of the variance in the dependent variable that is predictable from the independent variables
  - Measures model accuracy, explaining variance between predicted and actual
- $R^2 = 1 - \frac{\sum_{i=1}^{N}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{N}(y_i - \bar{y})^2}$

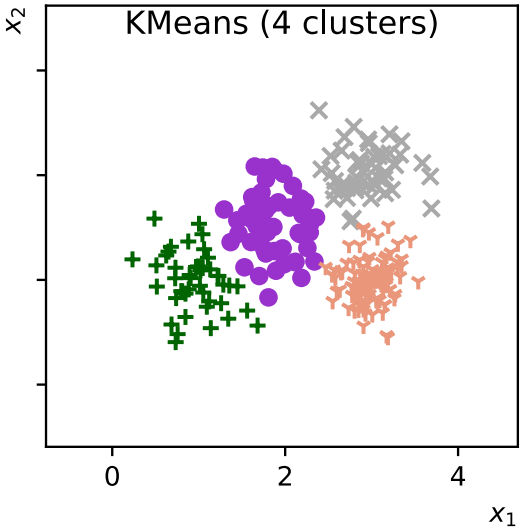
Unsupervised Learning

- Clustering used for exploratory data analysis to find grouping(s) or hidden patterns
- Clusters are identified using metrics, such as Euclidean distance, set similarity
- The algorithm groups the data that have not been labeled, classified, or categorized
  - Identifies commonalities in the data
  - It reacts based on the presence/absence of such commonalities in each new data
  - What is the "normal" in the dataset? What is the baseline?
  - Which data points are abnormal or anomalous in the dataset, if there are any?
  - How many clusters/densities are there naturally in the dataset?
  - Is there a probability distribution that can model the input dataset?
  - Can we find their density parameters?

Unsupervised Learning

- Euclidean distance
  - $\mathrm{d}_{ik} = \sqrt{\sum_{j=0}^{M}\left(x_{ij} - c_{kj}\right)^2}$
- Manhattan distance
  - $\mathrm{d}_{ik} = \sum_{j=0}^{M}\left|x_{ij} - c_{kj}\right|$
- Cosine similarity
  - $\mathrm{d}_{ik} = \cos(\theta) = \frac{x_i \cdot c_k}{|x_i||c_k|}$
- Jaccard index
  - $\mathrm{d}_{ik} = \frac{x_i \cap c_k}{x_i \cup c_k}$





Dimensionality Reduction with PCA

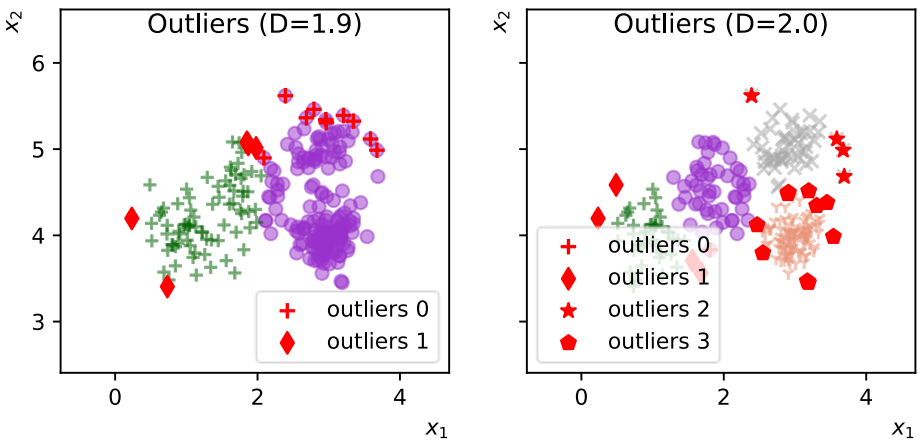
- Reducing the data dimensions helps in various ways
  - Faster method run-time, such as training
  - More generalized models decrease overfitting
  - Simpler models liked better by the domain expert or subject-matter expert (SME)
  - Noise reduction
- PCA projects data onto a new set of orthogonal axes (principal components) that capture the most variance
  - Preserves global structure but may lose interpretability in high dimensions (lossy)
  - Commonly used for reducing dimensionality before other analyses or visualization

PCA Algorithm

- Let $\mathbf{X} \in \mathbb{R}^{n \times m}$ is the dataset with n samples and m features
- **Mean centering** subtract the mean of each feature from the data
  - $\mathbf{X}_{\text{centered}} = \mathbf{X}_{\text{c}} = \mathbf{X} - \mu$
- **Covariance matrix**
  - $\Sigma = \frac{1}{n-1} \mathbf{X}_{\text{c}}^{\text{T}} \mathbf{X}_{\text{c}}$
- **Eigenvalue decomposition**
  - $\Sigma v = \lambda v$
  - where v are the eigenvectors (principal components), and λ are the corresponding eigenvalues
- **Principal components** the eigenvectors corresponding to the largest eigenvalues represent the directions of maximum variance
- **Projection**
  - $\mathbf{X}_{\text{projected}} = \mathbf{X}_{\text{c}} \mathbf{V}_{\text{k}}$
  - $\mathbf{V}_{\text{k}}$ contains the top k eigenvectors
- This projection, $\mathbf{X}_{\text{p}}$, reduces the dataset to k dimensions while preserving the variance captured by the top components

Anomaly Detection

- Identifying rare items, events, or observations which raise suspicions by differing significantly from the majority of the data
- **Outlier Identification** finding data points that deviate significantly from the norm
- **Pattern Deviation** identifying patterns that differ from expected behavior, which can signal issues such as fraud, errors, or rare events



Reinforcement Learning (RL)

- Trial-and-Error search
- Learner decides which actions to take
- The need to balance **exploration** and **exploitation**
  - unfamiliar strategies and tried-and-true actions
- A goal-directed agent interacting with an uncertain environment
- Might yield a delayed reward
  - forgo short-term gains for greater long-term gains

- **Central Hypothesis** Agent goals can be described by the maximization of cumulative rewards
- Foundation of RL agent construction
  - Definition of (State, Action) pair
  - **State** current condition of the environment
  - **Action** decision agent takes given state
  - **Reward** as a result of an action taken by agent

The Bellman Optimality Equation

- The expected value of states and actions stored in a table called the Q-table $Q(s,a)$
- The Q-table indicates to the agent/learner the expected reward for action a in state s

$$\mathrm{Q}(s,a) \longleftarrow \mathrm{Q}(s,a) + \alpha\,(\mathrm{R}(s) + \gamma \max_{a'} \mathrm{Q}(s',a') - \mathrm{Q}(s,a))$$

- 0< α <1 is the learning rate parameter
  - closer to 1 reward recent actions
- 0< γ <1 is the discount factor
  - closer to 1 rewards future actions more
- $R(s)$ is the reward for state s

On-Policy vs Off-Policy

On-Policy Learning

Off-Policy Learning

Model Based vs Model Free Algorithms

Model Based

- The agent builds an internal model of the environment, capturing the dynamics of state transitions and rewards
- The agent uses this model to simulate and plan, enabling it to make more informed decisions

Model Free

- The agent learns a policy directly from the environment without explicitly modeling the dynamics of the environment
- The agent focuses on learning the optimal action-selection strategy based on the observed states and rewards

Reinforcement Learning Algorithms

- In most problems, the environment dynamics $p(s', r \mid s, a)$ is not known
- Monte Carlo, temporal difference learning, Q-learning, and deep Q-learning approaches are the solutions
- SARSA State–action–reward–state–action

Method	Notes
Dynamic Programming	Assumes environment dynamics are known
Monte Carlo	Environment dynamics are not available Interacts with the environment
Temporal Difference	Improvement over MC Updating the value functions after each step
SARSA	On-policy
Q-Learning	Off-policy

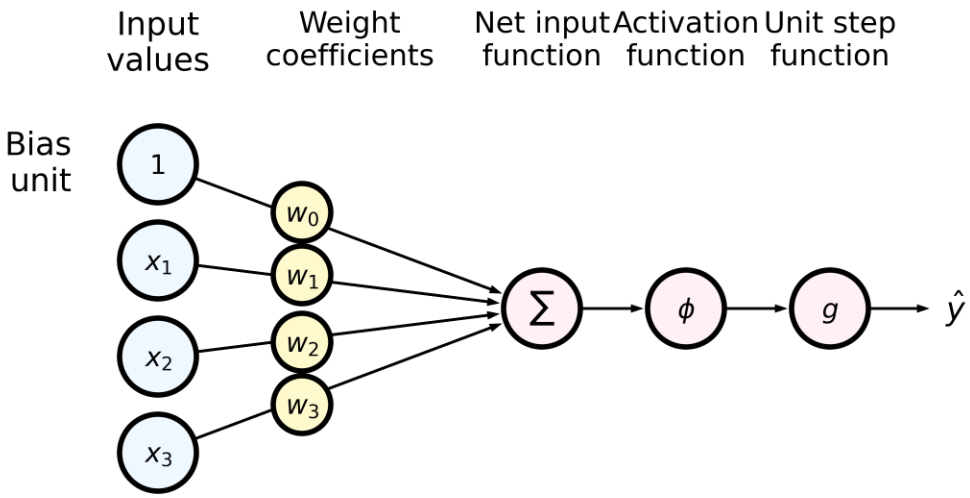
Neural Networks and Deep Learning

JOHNS HOPKINS
WHITING SCHOOL of ENGINEERING

Neural Networks

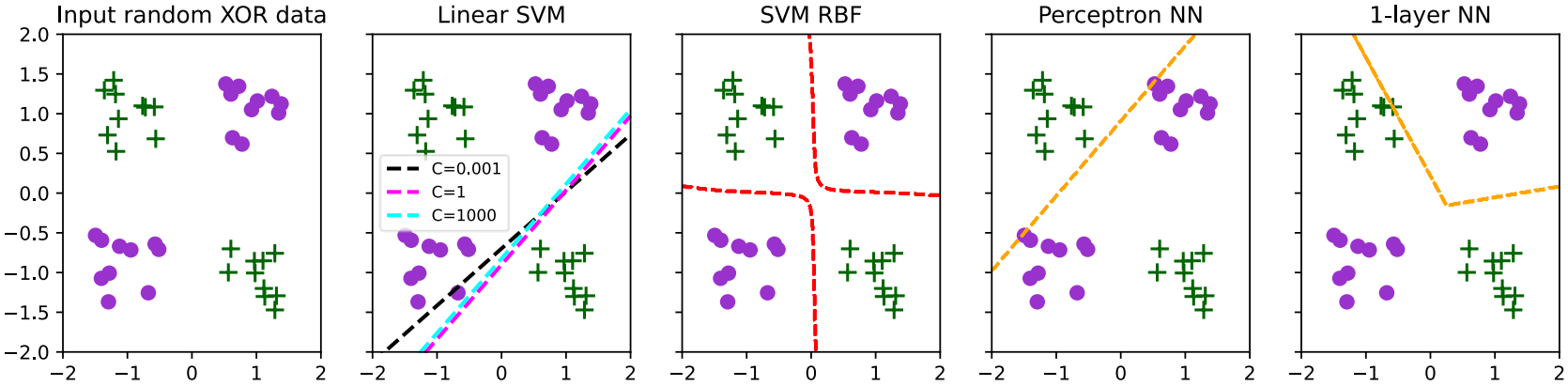
- The perceptron is a binary classifier that attempts to find an M dimensional hyperplane **w** defined by $\mathbf{w} \cdot x = \mathbf{w}^{\mathrm{T}} x = b$ where $x, \mathbf{w} \in \mathbb{R}^M$, and $b \in \mathbb{R}$
- The hyperplane separates the data points **X** into two categories {0,1}, originally provided by the **y** vector (ground truth)
- $\mathbf{f}(x) = \hat{\boldsymbol{y}} = \begin{cases} 1 & \text{if } \mathbf{w} \cdot x + b > 0 \\ 0 & \text{otherwise} \end{cases}$
- Finding out the hyperplane or w, the perceptron training algorithm, such as Adaline, updates w by iteratively computing the classification error
- $\mathbf{w} \leftarrow \mathbf{w} + \Delta\mathbf{w}$, where $\Delta\mathbf{w} = -\eta \nabla \mathrm{J}(\mathbf{w})$ and $\nabla \mathrm{J}(\mathbf{w})$ is the cost function and η is the learning rate

Single-layer Single-cell Neural Network



Neural Network

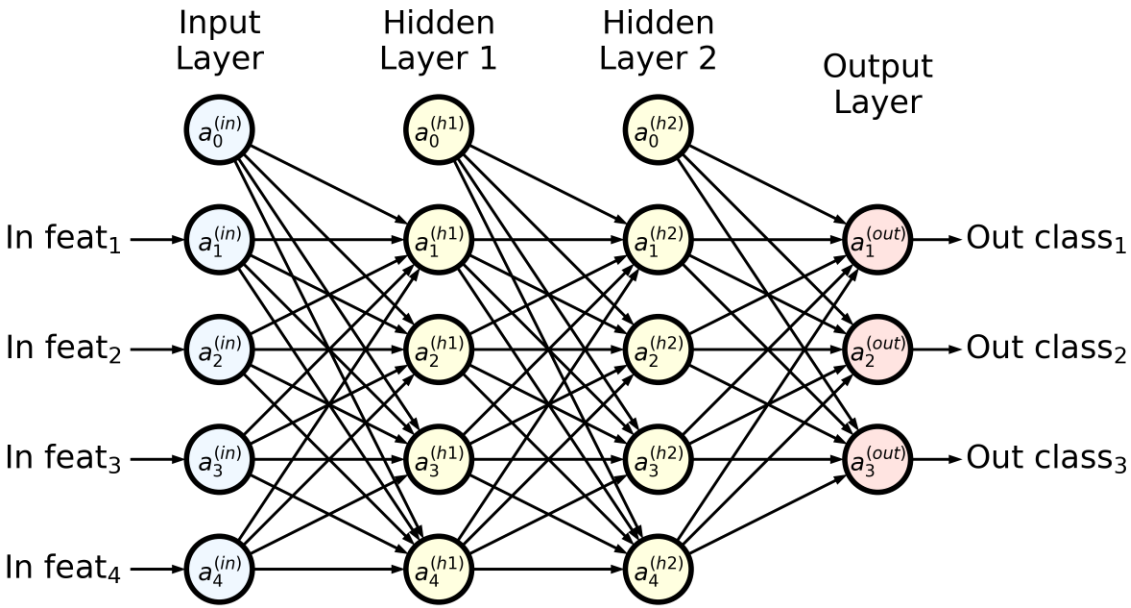
- Since a perceptron uses a single hyperplane, it cannot solve the **XOR** problem because it is not linearly separable
- An **XOR** problem is presented in the plot below, where two classes reside in opposing quadrants



Multilayer Neural Network Architecture

- In a multilayer neural network structure, the input is weighted and summed at one or more hidden layers representing a directed graph
- When each layer output is input to the next layer, this architecture is called **feedforward** neural network
- When the output of a layer is **fed-back** to a previous layer then it is called **recurrent** neural network in which from a time step to the next, some neuron layers will remember information they had in the previous time-step
  - This gives the neuron the ability to work as a memory cell
- The input vector $a^{(in)}$ has a bias value, $a^{(in)} = \begin{bmatrix} 1 \\ x_1^{(in)} \\ \vdots \\ x_m^{(in)} \end{bmatrix}$
- The output vector is one-hot encoded categories

Multilayer Neural Network



How does an NN Learn?

- **Initialize** the NN with random weights/biases
- **Forward Pass** (inference) data through the input, hidden, and output layers
- **Calculate Error** using error or loss function at the output
- **Backpropagation** involving the adjustment of the weights/biases to minimize error, utilizing optimization algorithm, such as gradient descent
- **Repeat** to achieve convergence

- **Strengths of an NN**
  - Representation learning
  - Scalability
  - Adaptability
  - Non-linearity
  - Parallel processing
  - Regularization
  - Availability of data
- **Drawbacks**
  - Require large datasets
  - Significant compute needed
  - Complex to design and tune
  - Lack interpretability

Deep Learning Notes

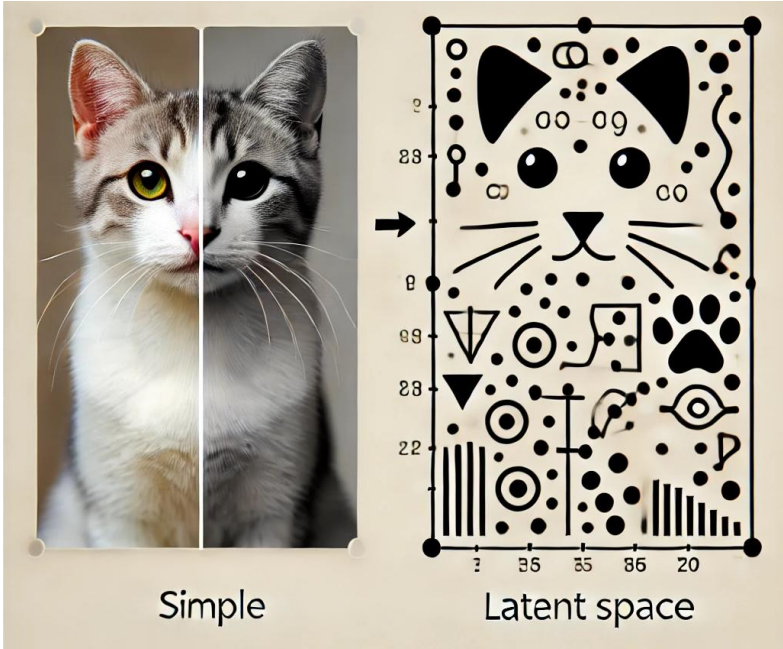
- Any neural network with two or more hidden layers are considered as a **deep NN**
- Very deep networks, such as ResNet (with over 100 layers), excel in tasks like image recognition (e.g., ImageNet competition) by capturing more complex hierarchical features compared to shallow networks
  - Deeper layers progressively learn more abstract representations, but training can become challenging due to vanishing gradients
  - Longer networks might suffer from vanishing or exploding gradients
- Techniques like residual connections in ResNet and highway networks mitigate this
- In shorter networks (e.g., simple Convolutional Neural Networks with 3-4 layers), the model captures basic patterns (like edges in images)
- Very deep networks can capture more intricate patterns (e.g., facial features or object parts) due to their depth and hierarchical structure

Autoencoders

JOHNS HOPKINS
WHITING SCHOOL of ENGINEERING

Autoencoders

- A type of neural network to efficiently represent input data without labels
- Generates an "n-layer" coding of the given input to reconstruct the input by identifying key data features
- The **latent space** retains the compressed data representations and captures the most important features/distribution of the input
- An autoencoder is a regression task that models an identity function

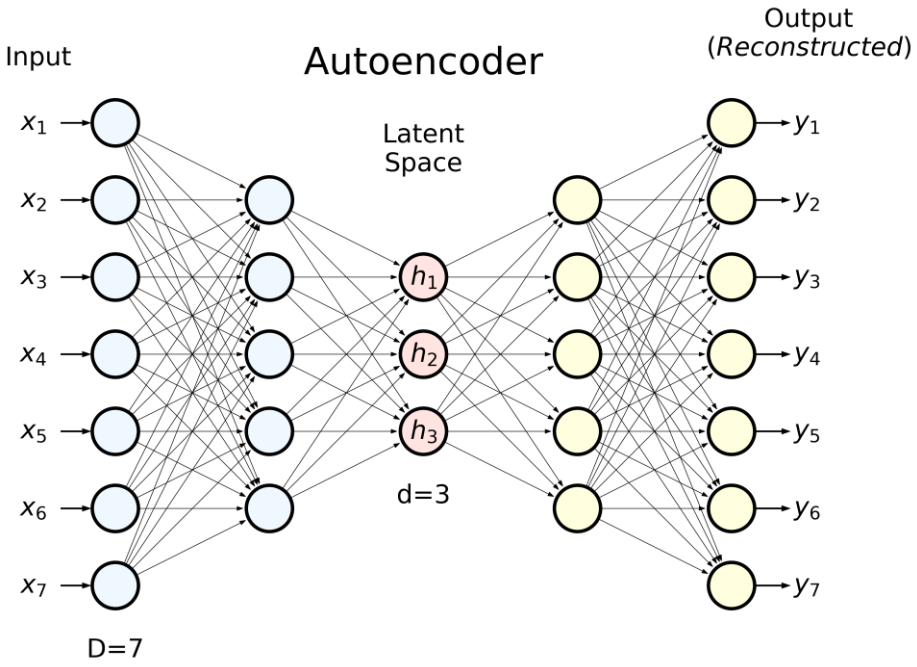


Autoencoder

- An encoder **f** maps from $\mathbf{x}$ to a representation $\hat{\mathbf{z}}$
- A decoder **g** maps from $\hat{\mathbf{z}}$ to observed data $\mathbf{x}$
- The model is trained to $\mathbf{x} \approx g(f(\mathbf{x}))$
- Minimize the error

$$\sum_j \left\| x_j - g\left(f\left(x_j\right)\right) \right\|^2$$

- A low-dimensional $\hat{\mathbf{z}}$ will retain as much information as possible
- Closely connected to PCA



Importance of Latent Space

- Latent space enables the model to perform **dimensionality reduction** by capturing the underlying structure of the data with fewer dimensions
  - This is essential in tasks like compression, denoising, and anomaly detection
- The most informative representation of the input is captured and redundant or less relevant information is discarded which is a form of automatic **feature extraction**
- By constraining the dimensionality of the latent space, autoencoders effectively **regularize** the learning process, preventing the network from simply memorizing the input data and forces it to learn **general patterns** or features
- Latent space often can reflect the **intrinsic manifold** on which the data lies, so the autoencoder's optimized latent space typically aligns with this manifold

Variational Autoencoders (VAEs)

- Variational Autoencoders (VAEs) are a type of generative model that combine principles from both probabilistic graphical models and neural networks
- If an embedding (or a latent vector) falls in the middle of two clusters, then that particular data point might be represented by an incorrect feature vector
- Variational autoencoders (VAE) help resolve this problem by generating feature vectors that have a 0-mean and 1-standard-deviation
- Bayesian reformulation
  - **Posterior** $q_\phi(\boldsymbol{z} \mid \boldsymbol{x})$, **Prior** $p(\boldsymbol{z})$, **Likelihood** $p_\theta(\boldsymbol{x} \mid \boldsymbol{z})$
- Evidence lower bound (**ELBO**)
  - $\log p(\boldsymbol{x}) \geq \mathbf{E}_{q_\phi(\boldsymbol{z}|\boldsymbol{x})}\left[\log p_\theta\left(\boldsymbol{x} \mid \boldsymbol{z}\right)\right] - \mathrm{KL}(q_\phi(\boldsymbol{z} \mid \boldsymbol{x}) \parallel p(\boldsymbol{z}))$

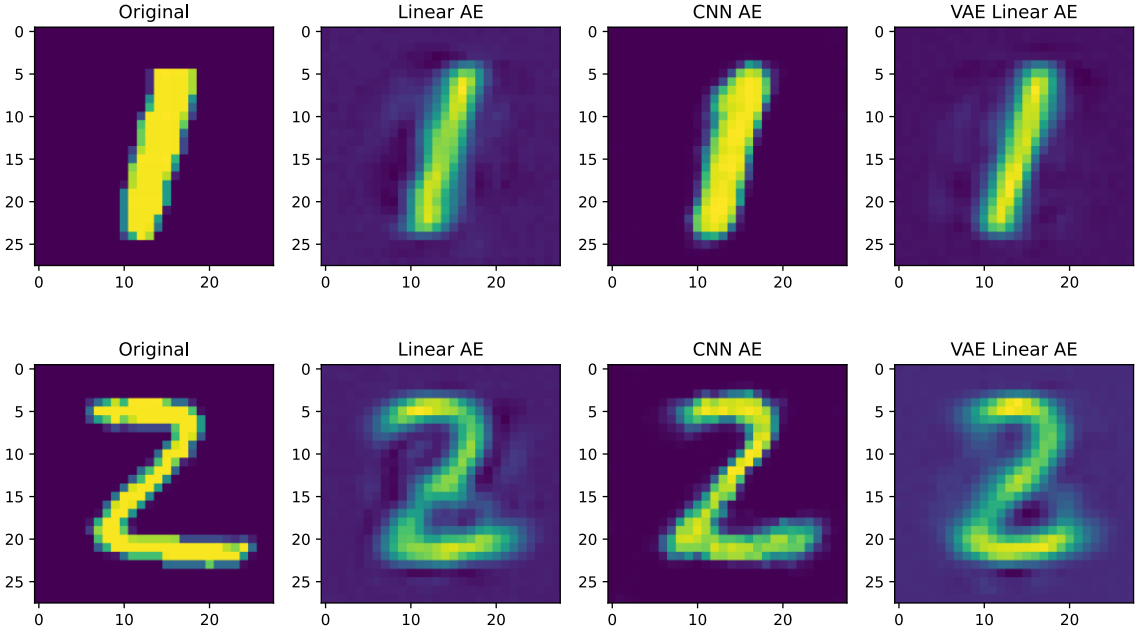
Variational Autoencoders (VAEs)

- The KL divergence tells how inefficient it is to use distribution Q to represent P
- Measuring how much information is lost when approximating P with Q

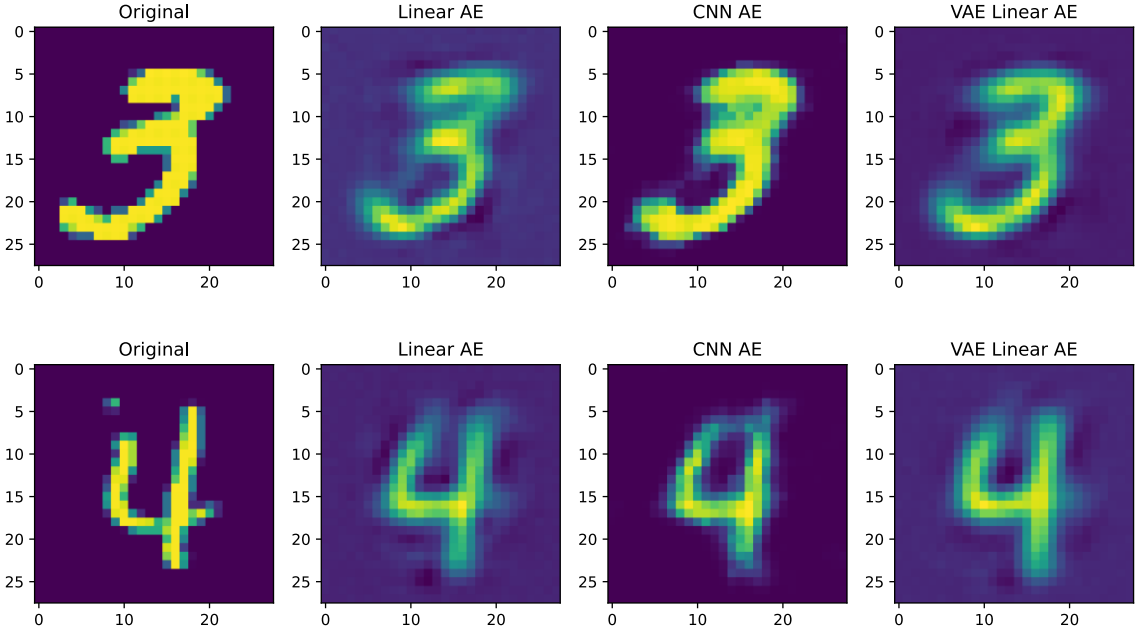
$$\mathrm{KL}(P \parallel Q) = \sum_{x} P(x) \log \frac{P(x)}{Q(x)}$$

- **Regularization** via $D_{\mathrm{KL}}(q_{\phi}(\boldsymbol{z} \mid \boldsymbol{x}) \parallel p(\boldsymbol{z}))$
- Autoencoder can be modified to be converted to VAE, define the latent space as,
- `z = mu + exp(log_var / 2) * epsilon`
- where `epsilon` is sampled from the standard normal distribution

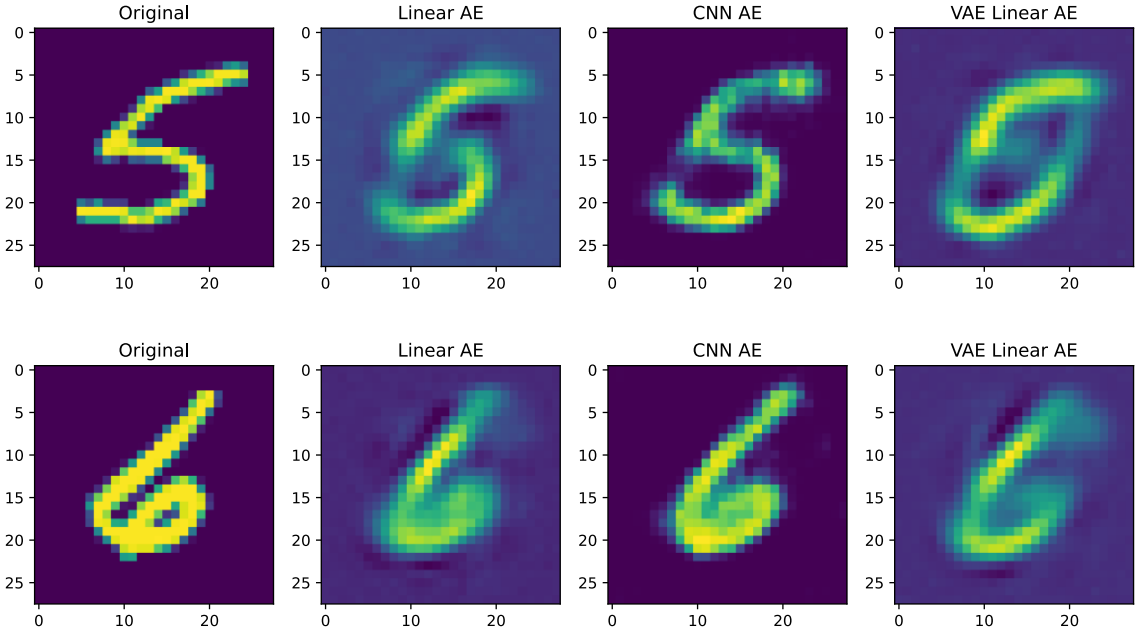
Autoencoder Output Examples



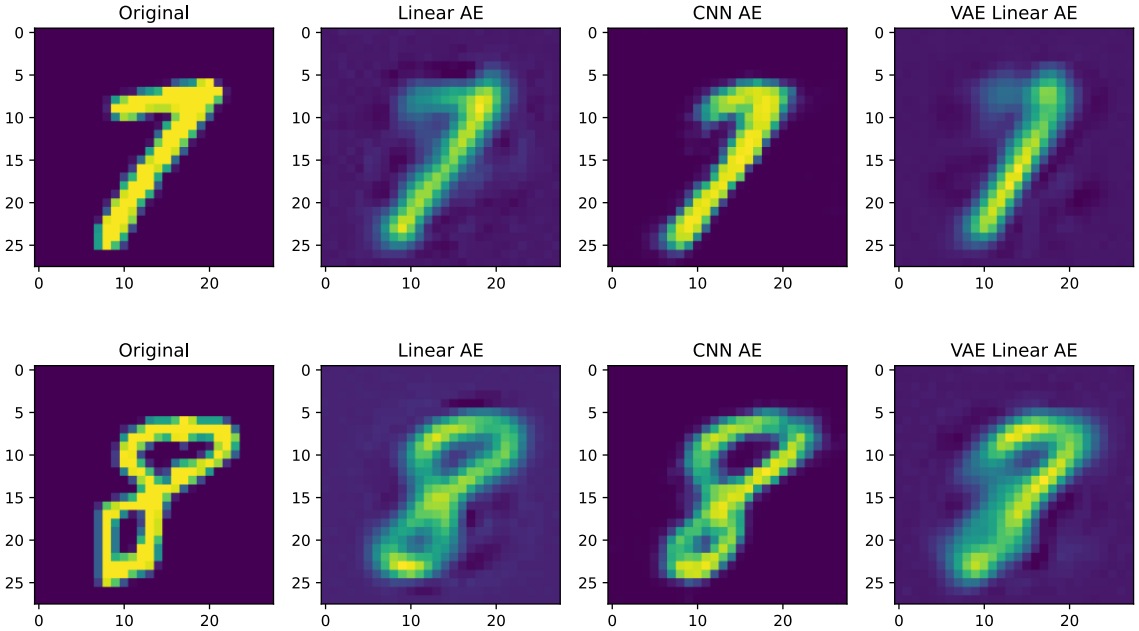
Autoencoder Output Examples



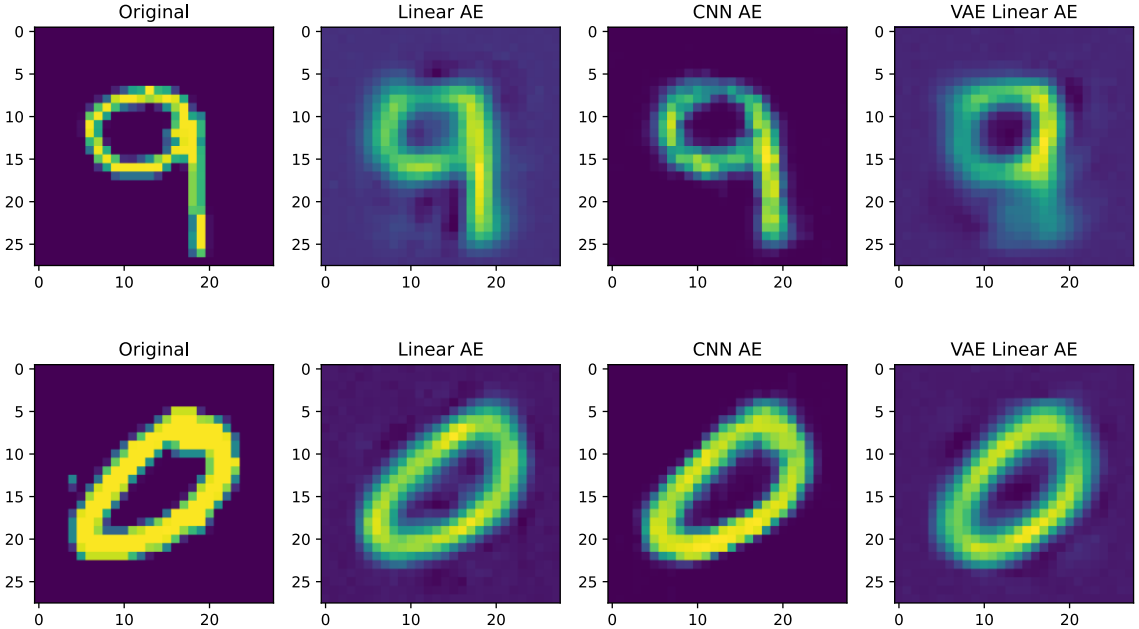
Autoencoder Output Examples



Autoencoder Output Examples



Autoencoder Output Examples



Variations

- Sparse Autoencoders
  - Sparse representation enforcement: $\| \boldsymbol{z} \|_0$ or $\| \boldsymbol{z} \|_1$
  - Sparse penalty term: $\mathrm{L}(\boldsymbol{x}, \widehat{\boldsymbol{x}}) + \lambda \| \boldsymbol{z} \|_1$
- Denoising Autoencoders
  - Corrupted input $\widetilde{\boldsymbol{x}}$: $\widetilde{\boldsymbol{x}} \sim q(\widetilde{\boldsymbol{x}} \mid \boldsymbol{x})$
  - Objective: $\min_{\theta_e, \theta_d} \mathbf{E}_{q(\widetilde{\boldsymbol{x}} \mid \boldsymbol{x})}[\mathrm{L}(\boldsymbol{x}, D(\mathcal{E}(\widetilde{\boldsymbol{x}})))]$
- Contractive Autoencoders
  - Jacobian regularization: $\Omega = \lambda \| \nabla_z \mathcal{E}(\boldsymbol{x}) \|_F^2$
  - Total loss: $\mathrm{L}(\boldsymbol{x}, \widehat{\boldsymbol{x}}) + \Omega$

Variations

- Stacked Autoencoders
  - Layer-wise unsupervised pretraining
  - Mathematical decomposition of features: $x \to z_1 \to z_2 \to \cdots \to z_n$
- Convolutional Autoencoders
  - Encoder/decoder structured with convolutional layers
  - Reconstruction loss in image domains: $\mathrm{L}(\boldsymbol{x}, \hat{\boldsymbol{x}}) = \| \boldsymbol{x} - \hat{\boldsymbol{x}} \|_2^2$ for pixel-wise accuracy
- Autoencoders for Dimensionality Reduction
  - Comparisons to PCA:
    - PCA minimizes reconstruction error $\| \boldsymbol{X} - \boldsymbol{X}\boldsymbol{W}^\top \boldsymbol{W} \|_F^2$
    - Autoencoders can model non-linear manifolds

Variations

- Undercomplete vs overcomplete autoencoders
  - **Undercomplete** $\dim(\boldsymbol{z}) < \dim(\boldsymbol{x})$
  - **Overcomplete** $\dim(\boldsymbol{z}) \geq \dim(\boldsymbol{x})$, regularized to prevent trivial solutions
- Adversarial Autoencoders
  - Regularization via GAN-like discriminator $D(\boldsymbol{z})$
  - Total loss: $\mathrm{L} = \mathrm{L}(x, \hat{\boldsymbol{x}}) + \alpha \mathbf{E}_{p(\boldsymbol{z})}[\log D(\boldsymbol{z})] + \beta \mathbf{E}_{q_\phi(\boldsymbol{z}|\boldsymbol{x})}[\log(1 - D(\boldsymbol{z}))]$
- Beta-VAE
  - Modified VAE objective:
    - $L_{\text{Beta-VAE}} = \mathbf{E}_{q_\phi(\boldsymbol{z}|\boldsymbol{x})}[\log p_\theta(\boldsymbol{x} \mid \boldsymbol{z})] - \beta D_{\mathrm{KL}}(q_\phi(z \mid x) \parallel p(\boldsymbol{z}))$
  - Control over disentanglement via β

Applications and Extensions

- Image denoising and inpainting
  - Reconstruction as a pretext task
- Anomaly detection
  - High reconstruction error for out-of-distribution data
- Sequence-to-sequence autoencoders
  - Mapping sequences using recurrent structures
- Generative models
  - Sampling from $p(\boldsymbol{z})$

References

- Foster, David. Generative deep learning. " O'Reilly Media, Inc.", 2023.
- Raschka, Sebastian, et al. Machine Learning with PyTorch and Scikit-Learn: Develop machine learning and deep learning models with Python. Packt Publishing Ltd, 2022.

JOHNS HOPKINS

WHITING SCHOOL
of ENGINEERING

© The Johns Hopkins University 2025, All Rights Reserved.